

Playwright for API Testing: A Comprehensive Guide

Introduction :

APIs have become essential building blocks in modern software development, allowing businesses to develop new products and services more efficiently by harnessing the capabilities of existing software and platforms. They serve as connectors, enabling different software systems, applications, and services to interact, share data, and perform functions. This opens up a wide array of possibilities for developers and businesses.

Playwright is one such framework that provides the ability to not only automate your UI flow, but it can also help you automate the APIs. Playwright added support for API testing from v1.16, making the perfect solution for E2E testing.

Why use Playwright for API Testing?

Playwright is primarily known as a tool for browser automation and end-to-end testing, but it can also be used for API testing. Here are some reasons why using Playwright for API testing might be advantageous:

Unified Testing Platform:

Playwright offers a unified platform for testing both web applications and APIs. This allows developers and QA engineers to use a single tool for different types of testing, reducing the need for multiple tools and simplifying the testing process.

JavaScript/TypeScript Support:

Playwright is built with JavaScript and TypeScript, which are commonly used languages in web development. Leveraging the same language for API testing as well as other testing tasks can streamline the testing workflow for developers familiar with these languages.

Cross-Browser and Cross-Platform Testing:

Playwright supports various browsers (Chrome, Firefox, WebKit) and can execute tests across different operating systems. This versatility allows for comprehensive API testing in diverse environments, ensuring compatibility and functionality across different setups.

Request Handling and Mocking: Playwright enables handling network requests, making it possible to intercept and modify HTTP requests and responses. This feature is beneficial for API testing as it allows developers to simulate different scenarios, test error handling, and mock responses, facilitating comprehensive testing.

Integration with Test Frameworks: Playwright can be integrated with popular testing frameworks like Jest, Mocha, or Jasmine. This integration allows for better test organization, reporting, and the use of various testing utilities that these frameworks offer.

Parallel Testing: Playwright supports parallel test execution, which can significantly reduce the overall testing time. This is particularly beneficial when dealing with a large number of API tests, improving efficiency and productivity.

Community Support and Updates: Playwright benefits from an active community and frequent updates. This ensures that the tool stays up-to-date with the latest web technologies, security measures, and best practices, making it a reliable choice for API testing.

While Playwright may not be the conventional choice for API testing, its flexibility and capabilities make it a viable option for teams looking to consolidate their testing efforts and leverage a powerful tool for various types of testing within the same ecosystem.

Pre-requisites :

To effectively use Playwright for API testing, several prerequisites should be considered:

Basic Understanding of JavaScript or TypeScript:

Playwright is primarily written in JavaScript and TypeScript. Familiarity with these languages is beneficial for creating test scripts and leveraging Playwright's functionalities effectively.

Installation of Node.js:

Playwright requires Node.js to be installed on the system. Node.js is a JavaScript runtime that allows you to execute JavaScript code outside of a web browser. You can download and install Node.js from its official website.

Installation of Playwright:

Once Node.js is installed, you need to install Playwright as a dependency in your project. You can use npm or yarn to install Playwright by running a command like `npm install playwright` or `yarn add playwright`.

Test Environment Setup:

Set up a test environment where you'll perform API testing. This could involve setting up a testing server or using existing APIs (like sandbox APIs) that you want to test.

Understanding of API Documentation:

It's essential to have a good understanding of the API endpoints, request methods (GET, POST, PUT, DELETE, etc.), request payloads, expected responses, authentication methods (if any), and error codes to write effective test cases.

Knowledge of Testing Framework:

While not mandatory, familiarity with testing frameworks like Jest, Mocha, Jasmine, etc., can be beneficial. Playwright can be integrated with these frameworks to manage and execute tests efficiently.

Understanding of API Testing Concepts:

Basic knowledge of API testing concepts, such as endpoint testing, parameterization, authentication testing, error handling, and response validation, is essential to create comprehensive test cases.

By meeting these prerequisites, you'll be better equipped to utilize Playwright effectively for API testing, ensuring a smoother testing process and reliable test results.

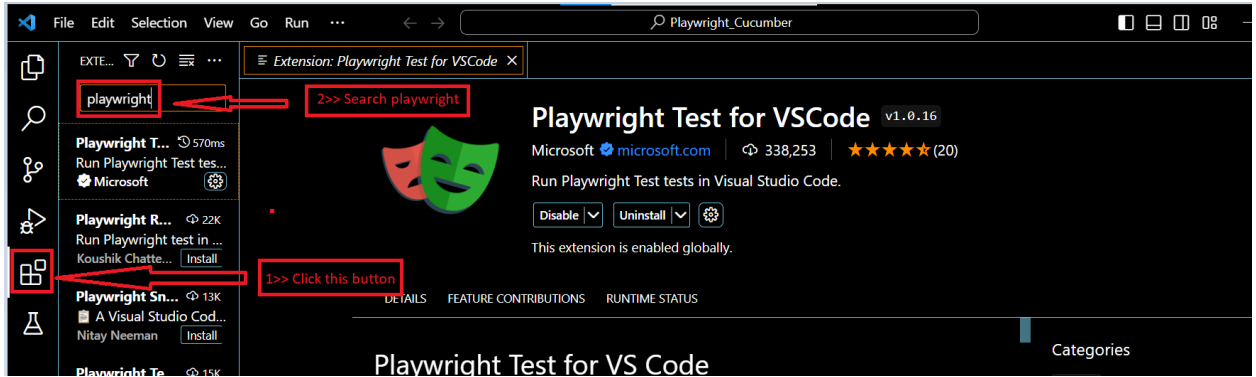
Installation and Setup of Playwright for API Testing :**Install through vscode :****Step 1 :**

- Download and Install the vscode to write and execute the script

Vscode download link : <https://code.visualstudio.com/download>

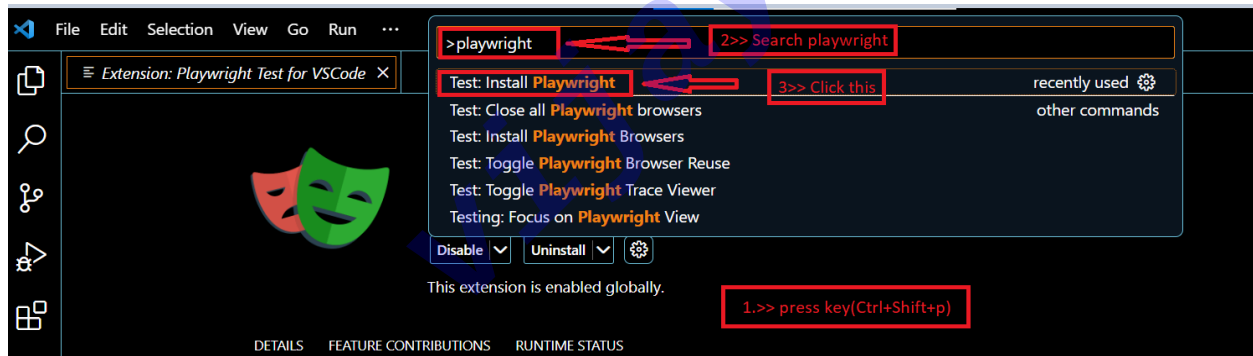
Step 2 :

- Create a new folder in VS Code.
- Click install after searching for "playwright test for vscode" in the Extension



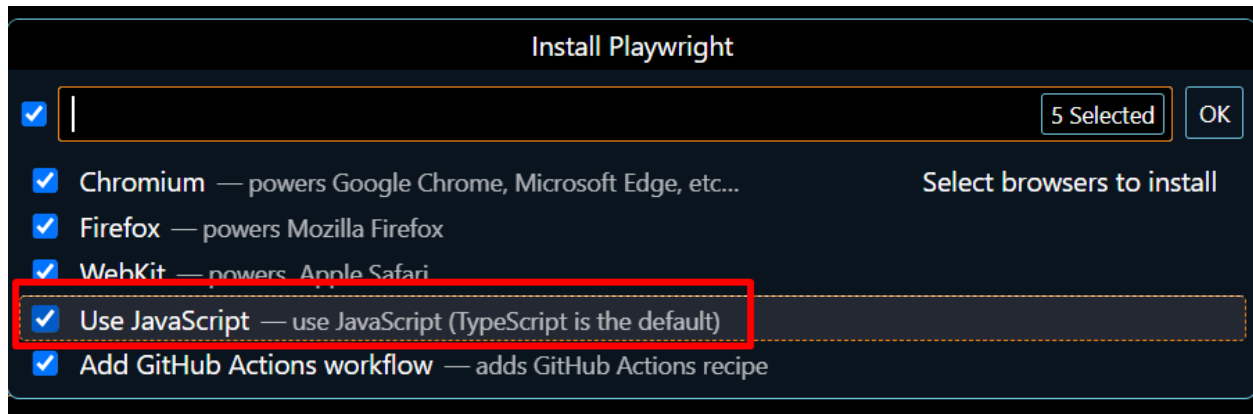
Step 3 :

- Search for playwright by pressing the keys (ctrl+shift+p)
- In this search, click on install playwright



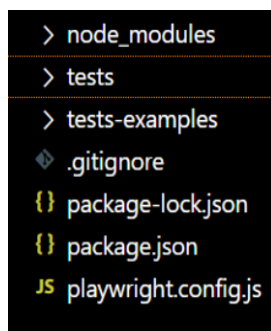
Step 4 :

- Select javascript in the check box, default is typescript



Step 5 :

- Playwright will download the browsers, as well as create the following files



- Check if the playwright installed or not , enter the command : **npx playwright --version**

How to write an API Test using Playwright :

Get :

Reading the newly generated user by utilizing an appropriate API endpoint and a GET request. Within the JSON body, insert the required parameters. We use the expect function to assert the responses.

```
import { test, expect } from '@playwright/test'

test('Get Request', async ({request}) => {
  const getRequest = await
request.get('https://reqres.in/api/users?page=2')
  const response = await getRequest.json()
  console.log(response)
```

```

console.log(response.data[0].name)
expect(getRequest.ok()).toBeTruthy()
expect(getRequest.status()).toBe(200)
expect(await response.data[0].first_name).toEqual('Michael')
expect(await response.data[0]).toHaveProperty("id", 7)
})

```

Post :

Using a post request to create new user data. When creating a user using a post request, the json body must contain the appropriate necessary data, such as name, email, etc. Put the user's required info into the authorization token. Provide accurate information that you gave along when writing your reply.

```

import { test, expect } from '@playwright/test'

test('Post Request', async ({ request }) => {

  const postRequest = await request.post('https://reqres.in/api/users', {

    data: {

      "name": "vijay",

      "job": "Trainee"

    }

  })

  const responseBody = await postRequest.json()

  console.log(responseBody)

  expect(postRequest.ok()).toBeTruthy()

  expect(postRequest.status()).toBe(201)

  expect(responseBody).toHaveProperty("name", 'vijay')

})

```

Put :

Updating the user with the proper API endpoint through a PUT request. Proper data, such as name, email, gender, etc., has to be given inside the json body when using the put request. After that, using the expect function to assert the request. If the request status is 200, it indicates that the put request was successful.

```
import { test, expect } from '@playwright/test'

test('Put Request', async ({ request }) => {

  const putRequest = await
request.put('https://reqres.in/api/users/560', {

    data: {

      "name": "Enter name",

      "job": "Admin"

    }

  })

  const responseBody = await putRequest.json()

  console.log(responseBody)

  expect(putRequest.ok()).toBeTruthy()

  expect(putRequest.status()).toBe(200)

  expect(responseBody).toHaveProperty("name", 'Enter name')

})
```

Patch :

PATCH requests are used when you want to modify only specific parts of a resource. It allows you to send the data that needs to be changed without resubmitting the entire resource.

```

import { test, expect } from '@playwright/test'

test('Patch Request', async ({ request }) => {

  const patchRequest = await
request.put('https://reqres.in/api/users/560', {

    data: {

      "name": "Ajay",

      "job": "Admin"

    }

  })

  const responseBody = await patchRequest.json()

  console.log(responseBody)

  expect(patchRequest.ok()).toBeTruthy()

  expect(patchRequest.status()).toBe(200)

  expect(responseBody).toHaveProperty("name", 'Ajay')

})

```

Delete :

Deleting a user with the appropriate API endpoint and a DELETE request. In order to delete a previously formed user, we must retrieve the correct ID from the user creation process and provide it to the DELETE request API endpoint. You can use the expect function from the chai library to provide appropriate assertions. If the status is 204, it indicates that the user's deletion was successful.


```
import { test, expect } from '@playwright/test'

test('Delete Request', async ({request}) => {

  const response = await request.delete('https://reqres.in/api/users/2')

  expect(response.ok()).toBeTruthy();

  expect(response.status()).toBe(204);

})
```

Mock API :

Mocking an API in Playwright involves intercepting and modifying network requests and responses in order to simulate the behavior of a real API without actually making requests to a server. This is incredibly useful in testing scenarios, where you might want to simulate different responses or test error handling without relying on a live API.

Playwright provides methods to intercept and mock network requests, allowing you to control the data returned by an API call within your test environment. The `page.route()` method in Playwright is commonly used to intercept network requests and provide custom responses.

Here's a basic overview of how you might mock an API in Playwright:

Intercept the Requests: Use `page.route()` or `context.route()` to intercept specific network requests made by your application. You can specify URL patterns to intercept or intercept all requests.

Provide Custom Responses: Once a request is intercepted, you can use `route.fulfill()` to provide a custom response. This includes setting the status code, response headers, and response body. You can return static data or dynamically generate responses based on your testing needs.

Testing Scenarios: This capability allows you to simulate different scenarios, such as testing for success, failure, edge cases, or slow network conditions, without relying on a live API. It's particularly useful for unit tests, integration tests, or end-to-end tests.

```
import { test, expect } from '@playwright/test'

test('mocks a fruit and does not call api', async ({ page }) => {
  // Mock the api call before navigating
  await page.route('*/**/api/v1/fruits', async (route) => {
    const json = [{ name: 'Strawberry', id: 21 }];
    await route.fulfill({ json });
  });

  // Go to the page
  await page.goto('https://demo.playwright.dev/api-mocking');

  // Assert that the Strawberry fruit is visible
  await expect(page.getByText('Strawberry')).toBeVisible();
});
```

Conclusion :

Playwright, though known for UI automation, offers robust capabilities for API testing. Its flexibility, language support, request handling, and integration options make it a viable choice for teams seeking to consolidate testing efforts within a unified ecosystem.

Overall, the guide provides a comprehensive understanding of leveraging Playwright for API testing, empowering teams to efficiently conduct API tests within their existing testing workflows.